TECHNICAL INTERVIEW PREP · 7 DE 10

Technical Interview Questions - Part 5 - Event Loop

Giuliana Olmos · 28 de noviembre de 2024 · https://dev.to/giulianaolmos/technical-interview-questions-part-5-event-loop-1ki4

Introduction

Hello, everybody!
Today, as the title says 😉, I'll be talking about the event loop.

This is not a topic that interviewers often ask about directly (I can only remember two occasions when they asked me to explain the event loop). BUT, in most interviews, they ask questions that are related to it. For example:

- "If I do this... what should be the behavior?"
- "If my code looks like this, what will be the output?"
- "Why does this code produce this output?"

All those questions are much easier to answer if you understand how the event loop works.

I'll be honest: this topic isn't my favorite. I much prefer questions about the behavior of code rather than explaining how the event loop works for 10 minutes straight.😅

Let's dive in! 😀

Questions

1. What is the event loop?
2. Examples

What is the event loop?

Short Answer:

> The event loop is responsible for handling asynchronous tasks in the JavaScript runtime.

To be honest, I don't think this answer is enough to satisfy the curiosity of an interviewer asking about the event loop. So, in this post, I'd like to dive deeper into this topic.

More than just knowing the concepts, it's important to understand **how** it works. That's why I've added some examples at the end.

Theory

What is the Event Loop?

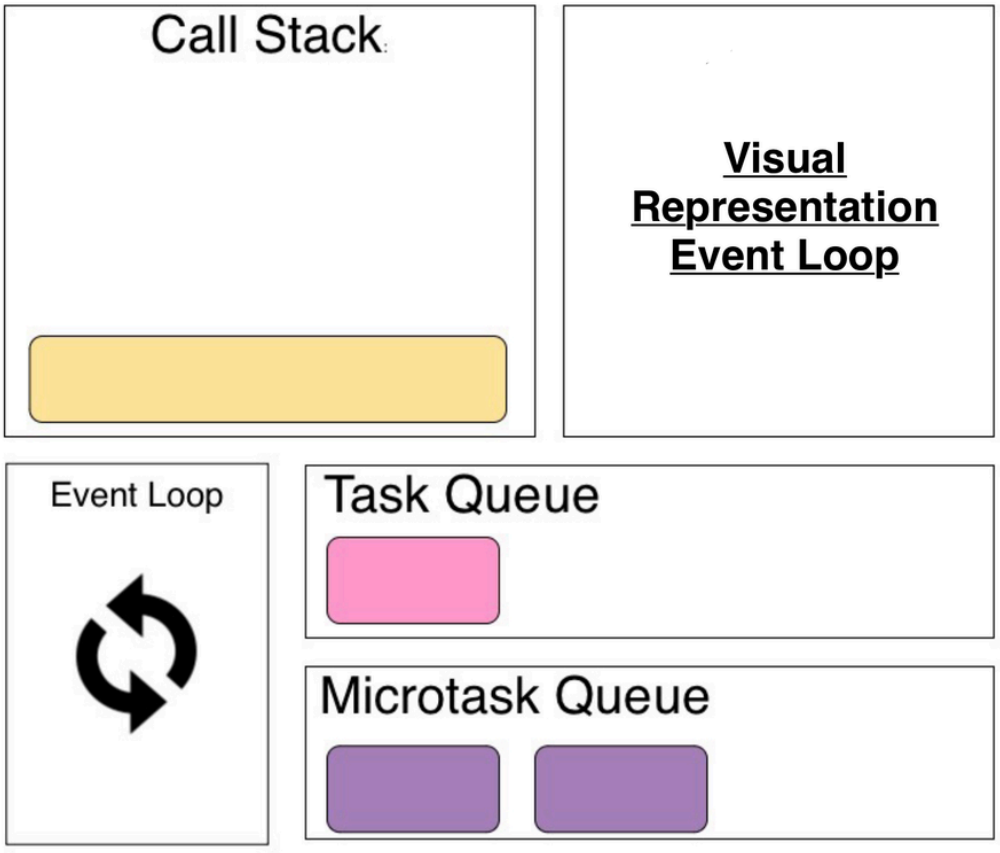
JavaScript has a runtime based on an event loop, which is responsible for handling tasks. Each language has a unique runtime, and an important point to note is that JavaScript is **single-threaded**.

What Does Single-Threaded Mean?

Single-threaded means that **JavaScript can handle only one task at a time**. This is why the event loop is so essential in JavaScript; it helps manage tasks efficiently despite this single-thread limitation.

Components of the Runtime

To understand the event loop better, let's first look at its main components:



Call Stack

The **call stack** is a data structure that keeps track of the functions we call. You can think of it like a stack of plates: when a function is called, it's added to the stack, and when it finishes, it's removed from the stack.

The call stack operates on a **LIFO (Last-In-First-Out)** principle, meaning JavaScript executes functions in the order they're stacked—from the topmost item down to the bottom, one at a time (remember, JavaScript is single-threaded).

Queues

In JavaScript's runtime, we have queues, which hold lists of tasks to be processed. Tasks in these queues wait until the call stack is empty.

- **Task Queue (or Callback Queue):** This queue stores tasks like `setTimeout()` and `setInterval()` calls. Tasks here are processed after the call stack is empty and after all tasks in the Microtask Queue have been processed. See more examples of tasks that are stored in this queue on MDN.
- **Microtask Queue:** This queue has priority over the Task Queue. It includes microtasks such as `Promise` callbacks and asynchronous functions like `process.nextTick()` and `async` functions.

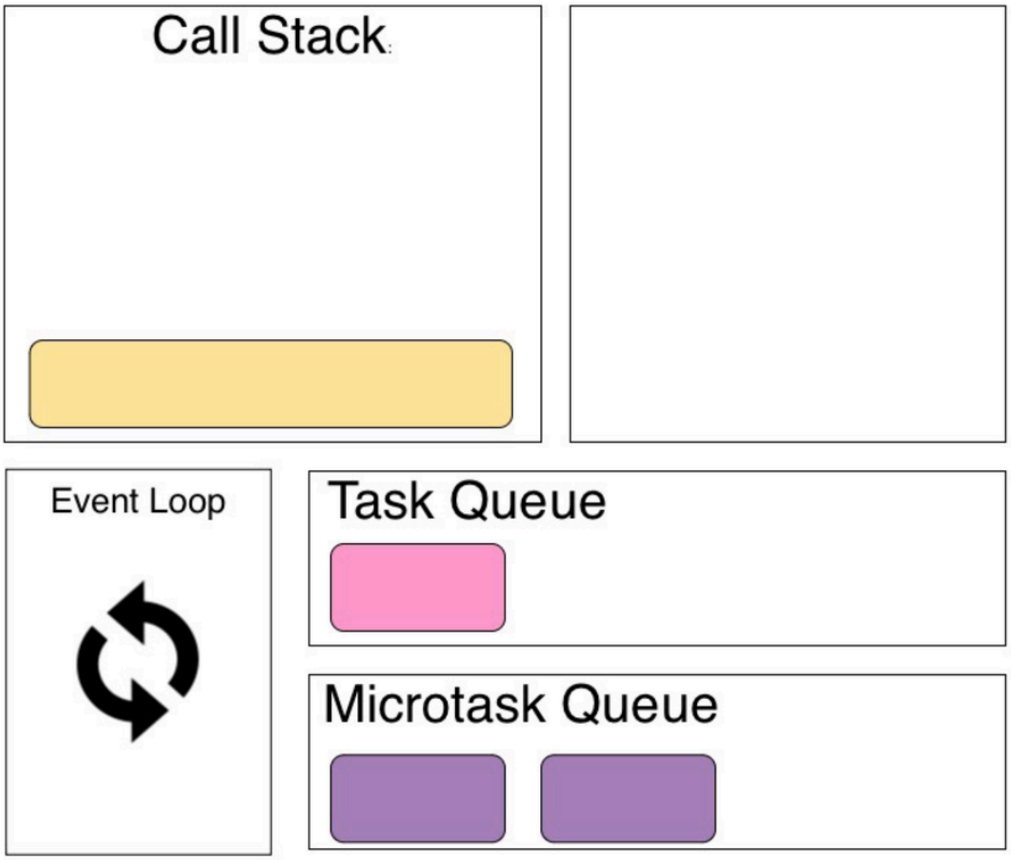
The **Task Queue** works on a **FIFO (First-In-First-Out)** basis, meaning tasks are processed in the order they're added, but only after the Microtask Queue is empty.

Event Loop

The **event loop** is a mechanism that manages the execution of asynchronous code. It observes the call stack and coordinates between the call stack and the queues (Task Queue and Microtask Queue) to keep the code running smoothly.

How Does It Work?

Let's go through the event loop process step by step. Refer to the image below for a visual representation.

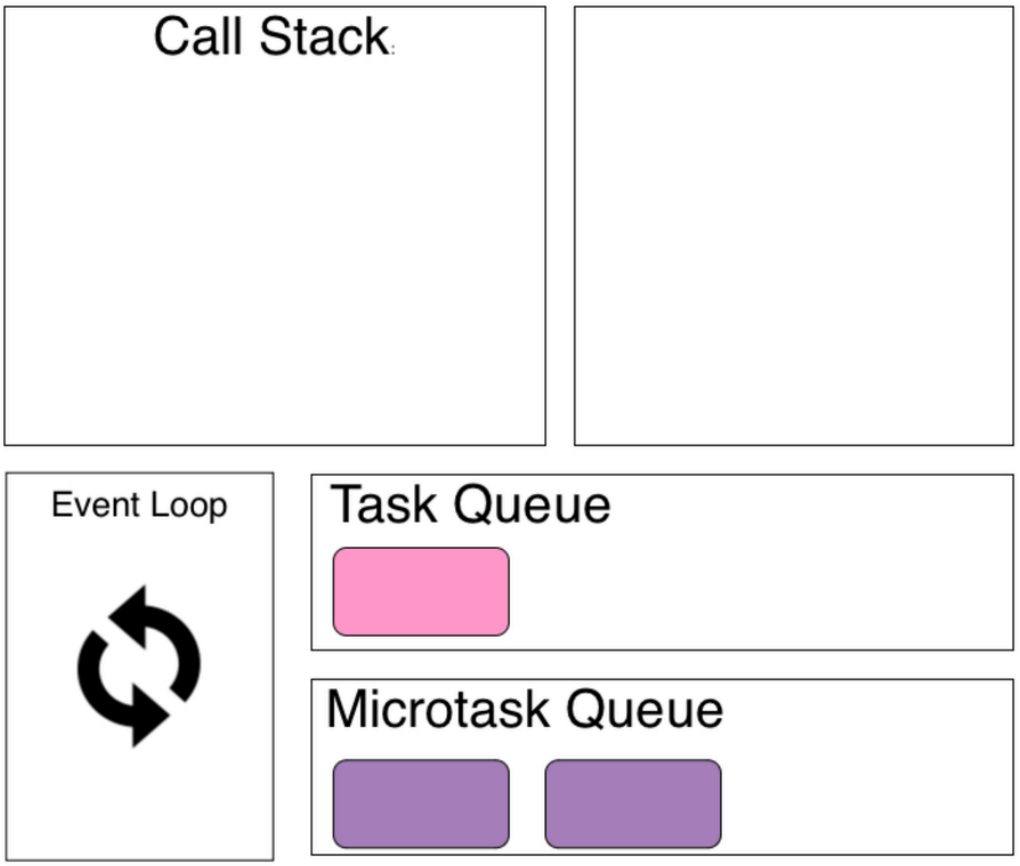


In this example:

- The **Call Stack** has one function.
- The **Microtask Queue** has two messages.
- The **Task Queue** has one message.

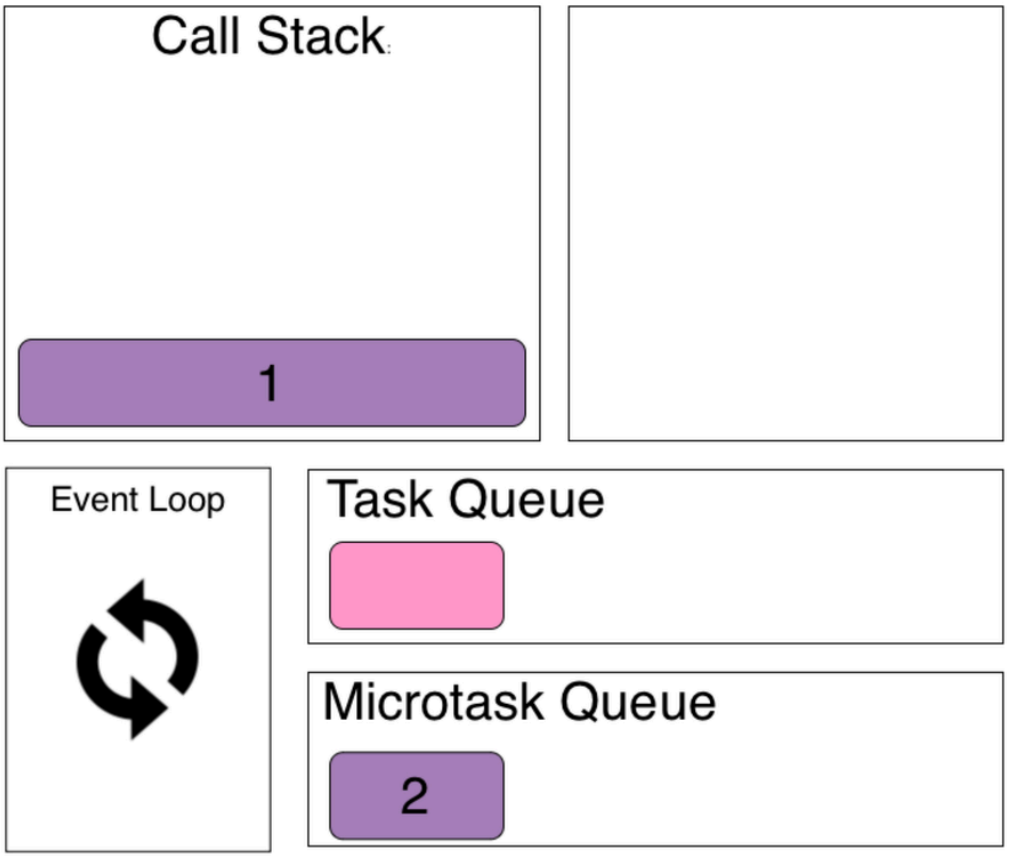
Step 1: Process the Call Stack

1. The event loop starts by looking at the **Call Stack**.
2. It finds a function in the stack and begins executing it.
3. Once this function completes, it is removed from the **Call Stack**.

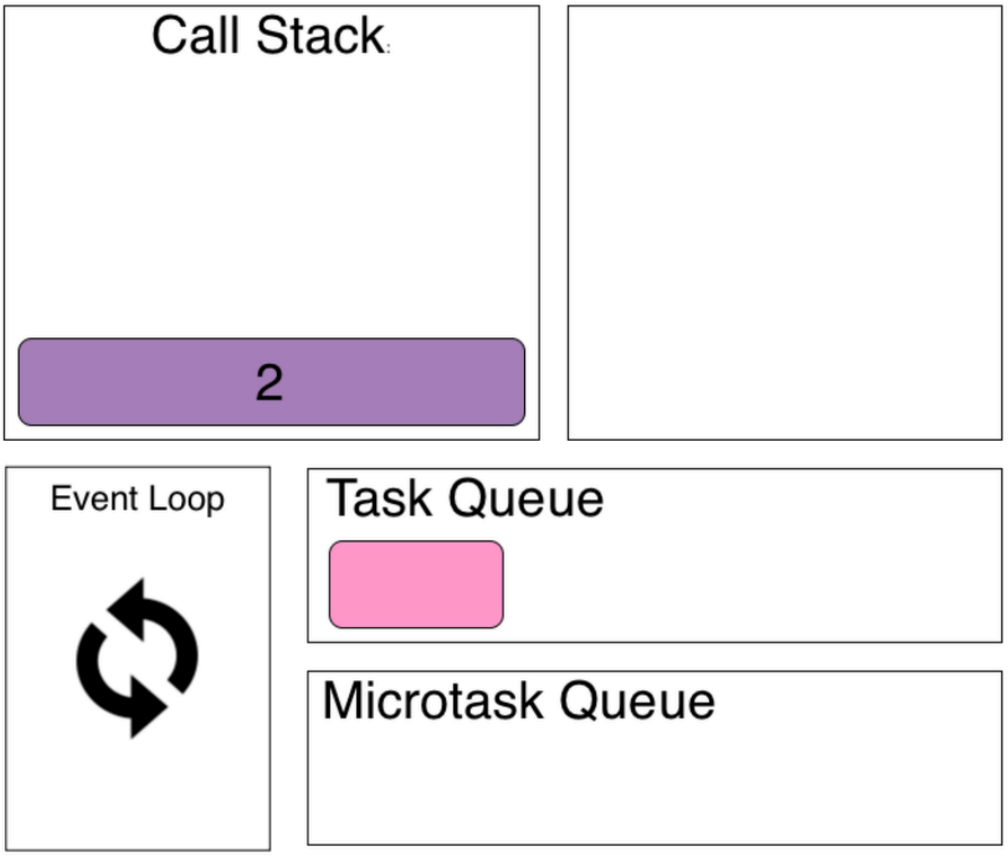


Step 2: Process the Microtask Queue

1. After the **Call Stack** is empty, the event loop checks the **Microtask Queue**.
2. It takes the first message from the **Microtask Queue** and pushes it to the **Call Stack** for execution.

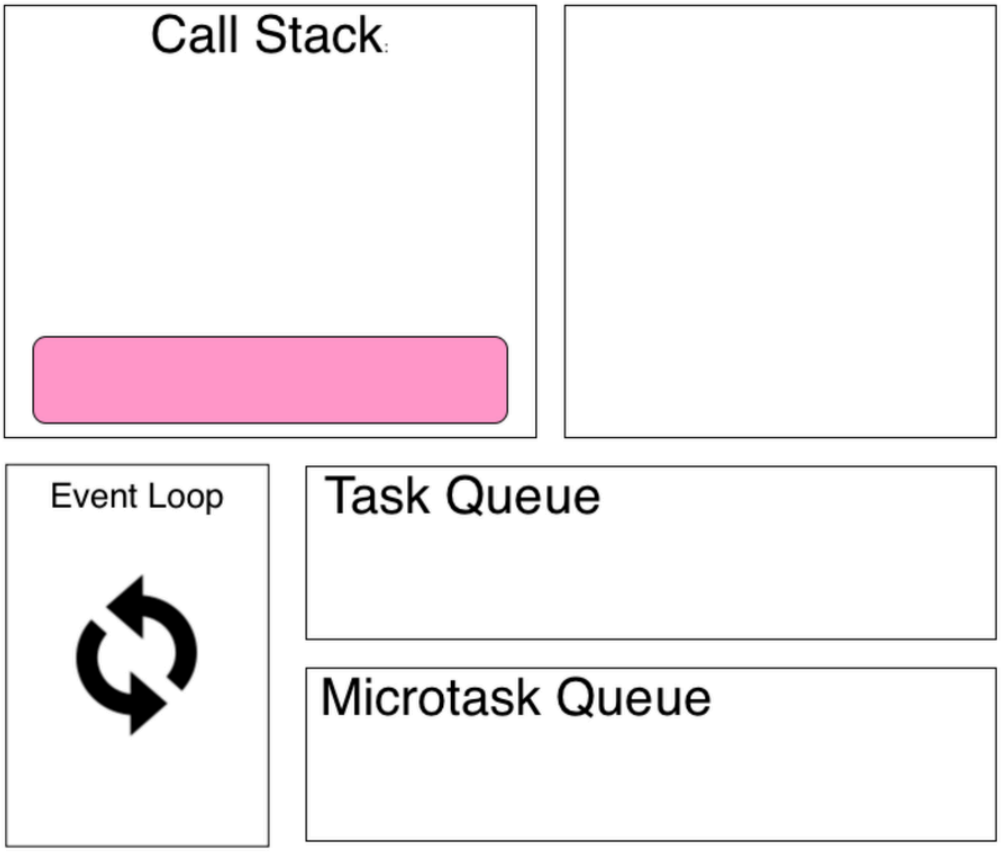


3. The function runs and, once completed, is removed from the **Call Stack**.
4. The event loop then moves to the next message in the **Microtask Queue** and repeats the process.
5. This continues until there are no more messages left in the **Microtask Queue**.



Step 3: Process the Task Queue

1. Once both the **Call Stack** and the **Microtask Queue** are empty, the event loop turns to the **Task Queue**.
2. It picks the first message in the **Task Queue** and adds it to the **Call Stack**.
3. The function runs, and when it completes, it’s removed from the **Call Stack**.
4. The event loop will continue this process with each task in the **Task Queue**, ensuring all tasks are handled one by one.



By following this order—**Call Stack**, then **Microtask Queue**, and finally **Task Queue**—the event loop helps JavaScript handle asynchronous code efficiently, even within its single-threaded environment.

Examples

Now that we understand how the event loop works and how tasks are prioritized, let’s look at some examples.

Example 1

```
const a = new Promise(function showA(resolve){
  console.log('A');
  resolve('B');
});

setTimeout(function showC() {
  console.log('C');
}, 0);

a.then(function showB(b) {
  console.log(b);
});

const d = function showD() {
```

```
  console.log('D');
};

d();
```

Before continuing, try to think about the order of the output.

✨What do you expect it to be?✨

Let’s break down each part of the code to understand why we get this output.

1. Creating the Promise

```
const a = new Promise(function showA(resolve) {
  console.log('A');
  resolve('B');
});
```

- Here, we create a new promise with a callback function.
- Inside this function, `console.log('A')` is executed immediately, so **"A"** is printed to the console.
- After logging "A", the promise is resolved with the value `"B"`.
- JavaScript recognizes there’s a `.then` callback (i.e., `showB`) that should run once the main call stack is clear, so it adds `showB` to the **Microtask Queue** (since promise resolutions go there).

2. `setTimeout` Call

```
setTimeout(function showC() {
  console.log('C');
}, 0);
```

- The `setTimeout` function schedules `showC` to run after 0 milliseconds.
- JavaScript places `showC` in the Task Queue because it’s a timer-based function.

3. `a.then` Callback

```
a.then(function showB(b) {
  console.log(b);
});
```

- This line registers a `.then` handler for the promise we already resolved in the previous step (`resolve('B')`).
- Since the promise is resolved, `showB` (the `.then` callback) is added to the **Microtask Queue**.

4. Defining `d`

```
const d = function showD() {
  console.log('D');
};
```

- This line simply defines the function `showD` but doesn't execute it yet, so nothing happens here.

5. Calling `d()`

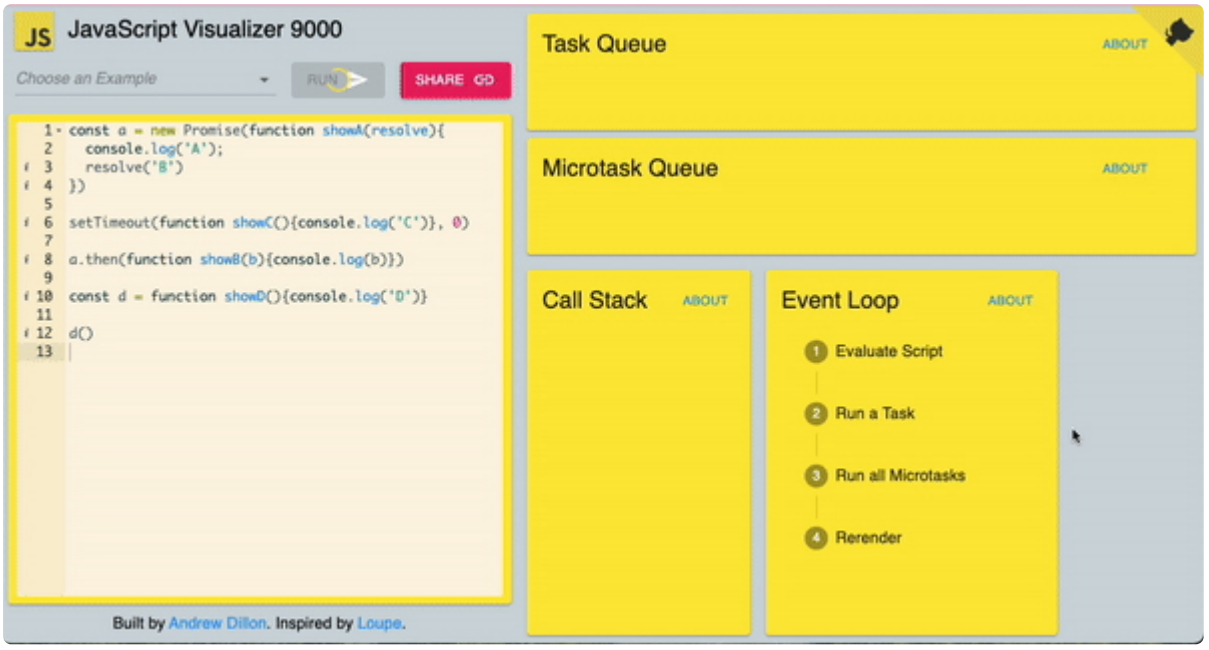
```
d();
```

- Now, we call `d()`, which is added to the **Call Stack** and executed. This results in `console.log('D')`, so **"D"** is printed to the console.

Final Output Order:

```
A
D
B
C
```

GIF for reference



Interactive Example

Example 2

```
console.log("Start!");

setTimeout(function showTimeout() {
  console.log("Timeout!");
}, 0);

Promise.resolve("Promise!")
  .then(function showPromise(res) {
    console.log(res);
  });

console.log("End!");
```

Again, take a moment to think about the order of the output.

✨What do you expect it to be?✨

Let's go with the explanation...

1. Logging "Start!"

```
console.log("Start!");
```

- This line is added to the **Call Stack** and executed immediately.
- As a result, **"Start!"** is printed to the console.

2. `setTimeout` Call

```
setTimeout(function showTimeout() {
  console.log("Timeout!");
}, 0);
```

- The `setTimeout` function schedules `showTimeout` to run after 0 milliseconds.
- JavaScript places `showTimeout` in the **Task Queue** since it’s a timer-based function.

3. Promise Resolution

```
Promise.resolve("Promise!")
  .then(function showPromise(res) {
    console.log(res);
  });
```

- The promise is resolved immediately with the value `"Promise!"`.

- JavaScript places `showPromise` (the `.then` callback) in the **Microtask Queue** because promises go into the microtask queue after being resolved.

4. Logging "End!"

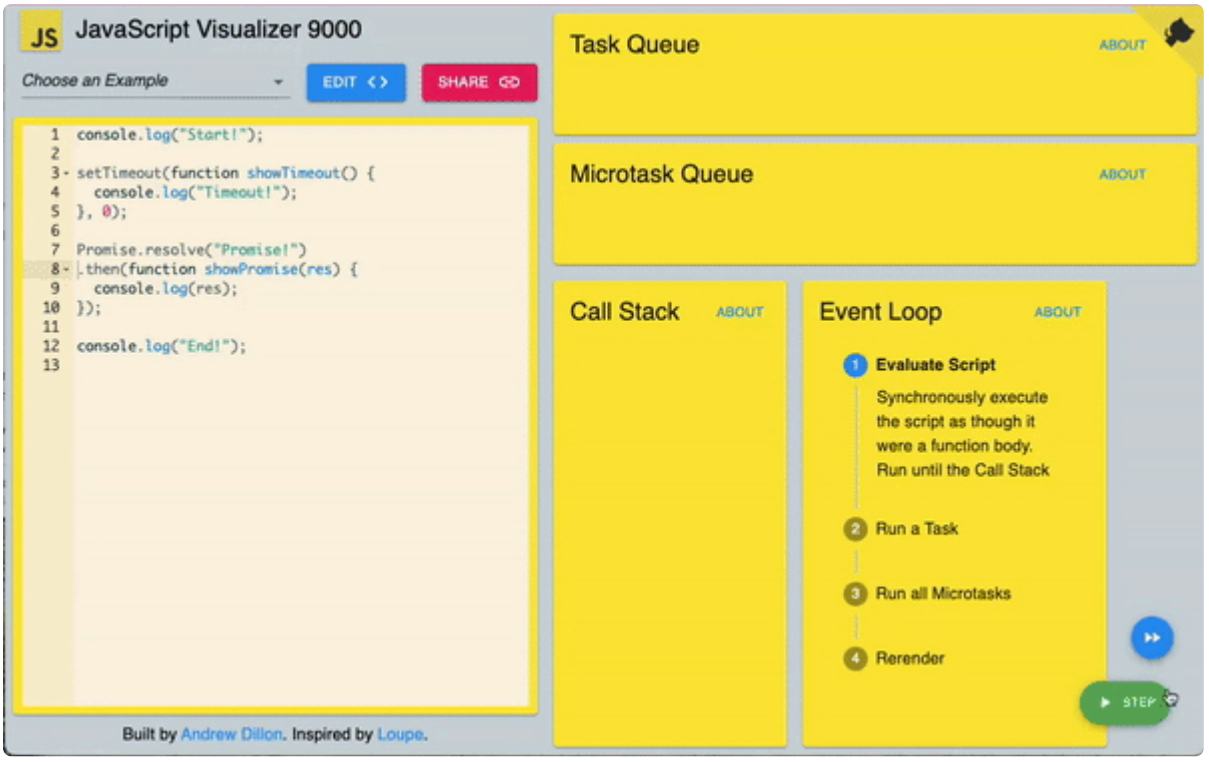
```
        console.log("End!");
```

- This line is added to the **Call Stack** and executed immediately.
- As a result, **"End!"** is printed to the console.

Final Output Order:

```
Start!
End!
Promise!
Timeout!
```

GIF for reference



Interactive Example

End

This chapter wasn’t too long, but I hope these examples helped you understand how the event loop works.

I strongly recommend experimenting with the interactive page to analyze other examples. Playing around on that page can make it much easier to understand the event loop in action.

Thank you so much for all the love on my previous posts!
See you next week! 😄

Bye Bye